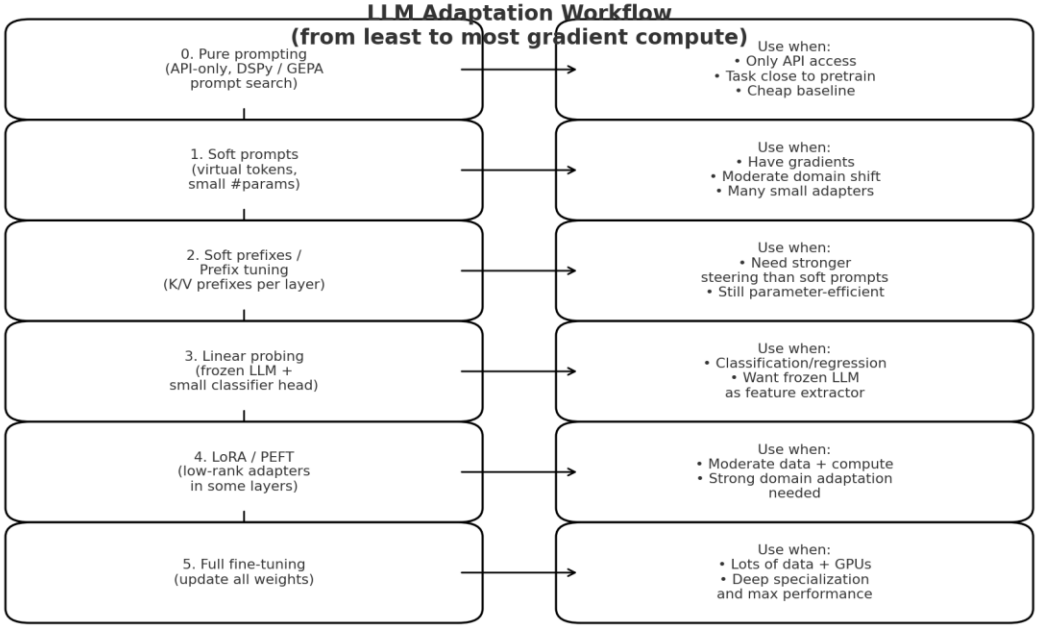
Using ChatGPT's Study Mode to Compare Different Fine-Tuning Strategies

Report Summary:

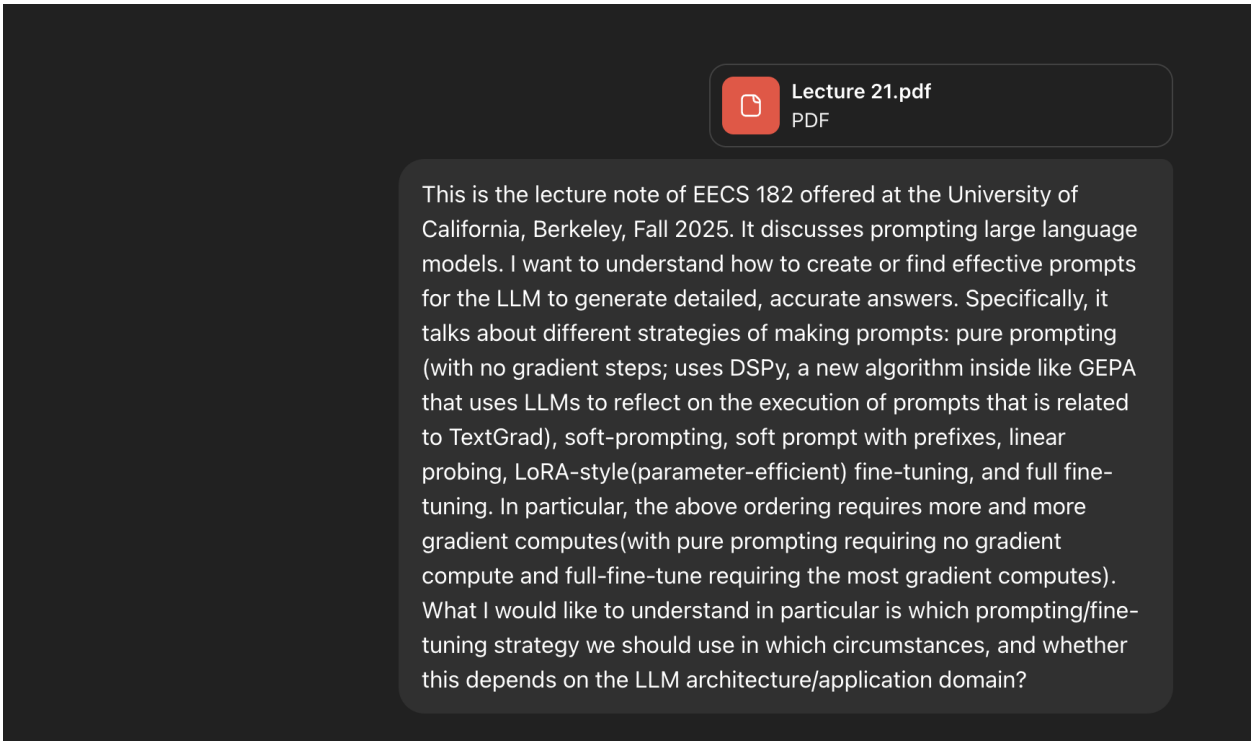
In this report, I use ChatGPT's study mode to compare a menu of fine-tuning strategies for adapting a pretrained large language model for a new task. It compares these strategies based on data access, model specificity, and application domains.

One feature of the study mode that I found very interesting is that it organizes concepts and aids the user's understanding by asking questions for each concept. Then, it moves to the next concept and repeats this Socratic tutoring behavior. Specifically, it provides a concrete example of applying a specific fine-tuning strategy. Then it creates an entirely opposite example to illustrate a different fine-tuning strategy.

Finally, it generates a nice workflow for applying fine-tuning strategies to get started on my own projects.



Prompt 1:


Lecture 21.pdf
PDF
This is the lecture note of EECS 182 offered at the University of California, Berkeley, Fall 2025. It discusses prompting large language models. I want to understand how to create or find effective prompts for the LLM to generate detailed, accurate answers. Specifically, it talks about different strategies of making prompts: pure prompting (with no gradient steps; uses DSPy, a new algorithm inside like GEPA that uses LLMs to reflect on the execution of prompts that is related to TextGrad), soft-prompting, soft prompt with prefixes, linear probing, LoRA-style(parameter-efficient) fine-tuning, and full fine-tuning. In particular, the above ordering requires more and more gradient computes(with pure prompting requiring no gradient compute and full-fine-tune requiring the most gradient computes). What I would like to understand in particular is which prompting/fine-tuning strategy we should use in which circumstances, and whether this depends on the LLM architecture/application domain?

ChatGPT’s answer:

Thought for 1m 11s ›

Great question—this lecture is basically giving you a *menu* of adaptation methods, ordered by “how much gradient compute / control” you use. Let’s turn that into a decision guide.

I’ll first give you the **axes to think about**, then go through **each method (0 → 5)**, and finally say **how architecture / domain interacts**.

(I’ll reference the note as we go. Lecture 21)

1. The 4 decision axes

When choosing between pure prompting, soft prompts, LoRA, full fine-tuning, etc., the main questions are:

1. **Do you control the weights?**
   - *Only API access* → you basically only have **pure prompting** and tools like **DSPy / GEPA / TextGrad** to search over prompts. Lecture 21
   - *You have the model checkpoint* → you can do soft prompts, LoRA, full FT, etc.
2. **How much labeled data + compute do you have?**
   - Tiny data / tiny compute → prefer methods with **few trainable parameters** (soft prompts, LoRA).
   - Lots of data + GPUs → full fine-tuning is on the table.
3. **How specialized is your task/domain?**
   - Close to pretrain (generic QA, summarization) → often **pure prompting / instruction tuning** is enough.
   - Far from pretrain (medical notes, code for a niche DSL) → you'll likely need **LoRA or full FT**.
4. **What output format?**
   - **Classification / regression** → linear probing (or small head) is very natural.
   - **Open-ended generation / dialogue** → prompt-based methods and generative fine-tuning (LoRA / full FT) are more natural.

Keep these axes in mind as we go through the ladder:

2. Strategy-by-strategy: what, when, why

0️⃣ Pure prompting (incl. DSPy / GEPA / TextGrad-style search)

What it is

- You **don't change any weights**; you only write better and better text instructions or few-shot examples.
- The lecture mentions tools like **DSPy** and algorithms like **GEPA** that use LLMs themselves to search for better prompts ("textual gradients", related to TextGrad). Lecture 21

When to use

- You **only have API access** to a model (no gradients).
- You want a **quick prototype** or to solve many tasks cheaply.
- Task is **close to what the model already knows** (general QA, summarization, basic coding).

Pros / cons

- ✅ Zero training cost; model stays general-purpose.
- ✅ Works with *any* LLM architecture (decoder-only, encoder-decoder, MoE, dense…).
- ❌ Ceiling on performance when the task is far from pretraining distribution.
- ❌ Prompt robustness can be fragile; performance may swing with wording.

1️⃣ Soft prompting (a.k.a. prompt tuning)

What it is

- You add a few **learned "virtual tokens"** to the input; only their **embeddings** are trainable, the rest of the model is frozen.
- In the lecture's numbers: embedding dim 4096, soft prompt length 100 → ~400K parameters vs 7–8B full model parameters. Lecture 21

When to use

- You have *some* labeled data and gradient access, but **want to keep the model mostly frozen** (e.g., many per-customer adapters).
- Domain is moderately shifted (e.g., legal Q&A, product support), but still primarily **text → text**.

Pros / cons

- ✅ Very parameter-efficient; you can store many task-specific prompts.
- ✅ Easy to swap between tasks; base model unchanged.
- ❌ Less expressive than changing internal weights; might not handle extreme domain shifts.

Architecture / domain notes

- Works best with **decoder-only** models (you just insert learned tokens at the front), but can be used with encoder/encoder-decoder as well.
- Great for **generation tasks** where you want to steer style/behavior.

2️⃣ Soft prompt with prefixes (prefix tuning)

What it is

- Instead of only learning extra embeddings at the input, you learn **prefix key/value vectors for the attention layers** (often across many layers).
- The lecture shows a "soft prompt prefix" that corresponds roughly to tuning the KV cache; with 32 layers, 100 prefix positions, and 4096-d keys/values you get ~26M parameters (still far under 7–8B).

  Lecture 21

When to use

- Soft prompting alone isn't enough; you need more capacity but still want **parameter efficiency**.
- Complex generation tasks: long-form dialogue style, specialized code generation, etc.

Pros / cons

- ✅ More expressive than simple soft prompts; can strongly change behavior.
- ✅ Still far cheaper than full FT; easy to have many task-specific prefixes.
- ❌ Training infra more complex (you must hook into all attention layers).
- ❌ Can still underperform full FT on very hard or very misaligned tasks.

Architecture / domain notes

- Originally popular in **encoder–decoder** models, but also used for decoder-only LLMs now.
- Especially useful for **sequence-to-sequence** formatting (e.g., summarization style, translation style).

3️⃣ Linear probing / feature extraction

What it is

- You treat the LLM as a **frozen feature extractor**, and train a *separate* small model (often a linear head) on top of the hidden states.
- This is item (1) in the lecture's "Step back: using a pretrained model for a new task" slide. Lecture 21

When to use

- Your downstream task is **classification, regression, or retrieval**, not generation.
- You have labeled data but **don't want to touch the base model** (e.g., you're calling it via API that exposes embeddings).

Pros / cons

- ✅ Very cheap to train; only a small head learns task boundary.
- ✅ Stable, easy to analyze; works great for many NLP benchmarks.
- ❌ Doesn't improve text *generation* behavior.
- ❌ Limited if the base representation doesn't already separate classes well.

Architecture / domain notes

- Especially natural for **encoder-only** models (BERT-style), but you can also use last-layer states from a decoder.
- Strong default for **text classification / clustering / semantic search**.

4️⃣ LoRA-style parameter-efficient fine-tuning (PEFT)

What it is

- You insert **low-rank trainable matrices** into some weight matrices (e.g., attention or MLP) and train only these; base weights stay frozen.
- Number of new parameters is usually in the **millions**, not billions.

When to use

- You have moderate data and compute, and need **stronger adaptation** than soft/prefix prompts.
- Domain or task is more different (e.g., domain-specific coding assistant, biomedical Q&A).
- You might need to **stack several adapters** (e.g., one per customer).

Pros / cons

- ✅ Much closer to full FT performance than soft prompting, with far fewer parameters.
- ✅ Widely supported in tooling; good trade-off between capacity and efficiency.
- ❌ More complex infra than prompt-only tricks; you still need gradient access and some GPU memory.
- ❌ If chosen poorly (wrong layers, rank too small) you might underfit.

Architecture / domain notes

- Works with most **Transformer architectures** (encoder, decoder, encoder–decoder, MoE) as long as you can hook into the weight matrices.
- Great for both **generation** and **classification** tasks.

5️⃣ Full fine-tuning

What it is

- You update **all weights** of the model (or nearly all).
- The lecture lists this as step (2): “adjust the weights of the pretrained model.” Lecture 21

When to use

- You have **lots of labeled (or high-quality synthetic) data** and plenty of compute.
- You need **maximum performance or strong domain specialization**:
  - training an InstructGPT-style chat model from a base LM,
  - retargeting a general LM to protein sequences, code only, a radically different language, etc.

Pros / cons

- ✅ Highest ceiling on performance and alignment.
- ✅ You can reshape the whole model: style, safety, capabilities.
- ❌ Very expensive in compute and engineering.
- ❌ Risk of **catastrophic forgetting** / losing general abilities if the data is narrow.
- ❌ Harder to maintain many different variants (each is a full new checkpoint).

Architecture / domain notes

- Used for all big families (BERT-style encoders, GPT-style decoders, MoE models, etc.).
- For MoE models, full FT can be especially heavy because there are many expert weights.

3. Does the *architecture* change the choice?

Mostly, the **choice is driven by access + data + domain**, not by small architectural differences. But there are some natural pairings:

- **Encoder-only (BERT)**
  - Linear probing or small-head fine-tuning for classification/extraction.
  - Soft prompts / adapters if used as a masked-LM-style generator.
- **Decoder-only (GPT, LLaMA, Qwen, etc.)**
  - Pure prompting and DSPy-style prompt search are very effective, since these models are trained autoregressively (exactly what prompts trigger). Lecture 21
  - Soft/prefix prompts, LoRA, and full FT are all standard for improving generation quality.
- **MoE vs dense**
  - Adaptation methods (soft prompts, LoRA, full FT) all *still work* for MoE, but you may need to be careful which experts/layers you adapt.
  - For many practical purposes, you can treat them similarly when picking a strategy.
- **Application domain differences**
  - **Classification, retrieval** → linear probing or LoRA on *top layers* often enough.
  - **Dialogue / tool use / agents** → you usually need **instruction-style full FT or LoRA**, not just prompts.
  - **Extreme domain shift** (bio, law, niche code) → LoRA or full FT; pure prompting usually underperforms.

4. A simple rule-of-thumb ladder

You can summarize the lecture's continuum like this (from least to most gradient compute):

1. **Pure prompting / DSPy / GEPA** – try this first when you only have an API or the task is close to generic capabilities.
2. **Soft prompts / prefixes** – when you need a *bit* of adaptation and can modify embeddings/attention, but want to keep params small. Lecture 21
3. **Linear probing** – for classification-style tasks using frozen representations.
4. **LoRA / PEFT** – for substantial improvements with moderate compute; default for serious domain adaptation on open models.
5. **Full fine-tuning** – for maximum performance and deep domain specialization when you own the model + hardware.

Quick check for understanding (your turn)

Suppose you:

- have **only API access** to a GPT-style model,
- want to improve performance on **multiple-choice QA**,
- and can evaluate thousands of prompt candidates but **cannot backprop**.

1. Which strategy from the list would you use?
2. Would you lean more toward **hand-designed prompts** or something like **DSPy/GEPA-style automatic prompt search**, and why?

If you answer those in 1–3 sentences, I can sanity-check and then we can walk through a contrasting scenario (e.g., "I have the weights and a GPU, want to build a medical assistant") and choose the best strategy there too.

Comment 1: A detailed explanation of the pros and cons of each fine-tuning strategy is provided, along with the application domain specification and architectural choices of LLMs. In addition, it asks me a question about a specific scenario to apply the most appropriate fine-tuning strategy.

Prompt 2:

I would choose pure prompting. I would use DSPy/GEPA-style automatic prompt search since the task (multiple-choice QA is general enough)

ChatGPT's answer:

Thought for 9s ›

Nice, that's exactly the move I'd make here 👍

- With **only API access**, you *can't* do gradient-based updates, so you're limited to **pure prompting**.
- Multiple-choice QA is very close to what big GPT-style models already see in pretraining, so the base model is already strong. The main problem is **finding good formats and instructions**, which is exactly what **DSPy / GEPA / TextGrad-style prompt search** is designed for: use the model itself (plus a validation set) to iteratively improve the prompt, no backprop needed.

So your reasoning is spot on:

> "General task + API-only + can evaluate many prompts → pure prompting with automatic prompt search."

Let's contrast with a different setup to cement the decision rules:

> New scenario: You have an **open-source 7B LLaMA-style model**, a **single 24GB GPU**, and **10–20k labeled examples** of domain-specific *free-form Q&A* (say, medical or legal). You want good **generation**, not just classification.
>
> In this case, which adaptation method would you reach for first:
> **soft prompts, LoRA-style PEFT, or full fine-tuning**—and why?

Try to pick one and give a 1–2 sentence justification based on:

- compute,
- number of trainable parameters,
- how far the domain is from pretraining.

Comment 2: It creates an exact opposite scenario for me to explain fine-tuning strategy choices.

Prompt 3:

Since the model size is big, with enough compute and training data, I should do fine0tuning for finest precision

ChatGPT's answer:

Good instinct that **more control → more precision**, but in this specific scenario I'd *actually* reach for **LoRA-style PEFT first**, not full fine-tuning. Let me unpack why.

Why LoRA > full fine-tuning here

Setup recap:

- 7B decoder model, **1 × 24GB GPU**
- **10–20k** domain-specific *free-form Q&A* examples
- Need **good generation** (not just classification)

Full fine-tuning:

- Needs **a lot of GPU memory + optimization tricks** to train 7B end-to-end on a single 24GB card (gradient checkpointing, low precision, etc.).
- 10–20k examples is *not that much* → high risk of **overfitting / catastrophic forgetting**, where the model becomes "too medical/legal" and loses general skills.
- Every time you want a new domain variant, you must store a **full 7B checkpoint**.

LoRA-style PEFT:

- You only train a few **low-rank adapters** (millions of params instead of billions).
- Much easier to fit and train on a single 24GB GPU.
- With 10–20k examples, you can usually reach **near–full-FT performance** while:
  - keeping the base model intact,
  - having small, swappable domain adapters (one for medical, one for legal, etc.).

So a good rule:

> **Moderate data + single GPU + decent domain shift → LoRA first;**
> **huge data + multi-GPU + mission-critical quality → consider full fine-tuning.**

Quick check for you

Given that intuition, can you fill in the blank?

> "If I have **100k–1M examples** and a **multi-GPU setup**, and I want the *best possible medical assistant*, I would probably move from LoRA to _____."

Just write the method's name and (if you like) one short reason.

Comment 3: It points out my mistake by specifying exactly where my logic is off. Full fine-tuning requires substantial GPU access and optimization tricks, and medium-sized examples can increase the risk of overfitting.

Prompt 4:

full-finetuning

ChatGPT's answer:

Yep, exactly ✅

> With **100k–1M examples + multi-GPU**, and you want the *best possible* medical assistant, you'd move to **full fine-tuning**.

Because:

- You now have **enough data** to safely adjust *all* weights without massively overfitting.
- You have **enough compute** that updating billions of parameters is feasible.
- The domain (medicine) is pretty far from generic web text, so it helps to let the model **reorganize its whole representation space** for that domain.

Super-short cheat sheet (tying it back to your original question)

From least → most gradient compute (roughly matching the lecture): Lecture 21

1. **Pure prompting (incl. DSPy/GEPA/TextGrad)**
   - *Use when*: API-only access, task close to pretraining, or you just want a fast baseline.
2. **Soft prompts**
   - *Use when*: you can train a tiny number of parameters and want per-task adapters, domain shift is moderate.
3. **Soft prefixes / prefix tuning**
   - *Use when*: soft prompts not strong enough; you want more expressive steering but still parameter-efficient.
4. **Linear probing / feature extraction**
   - *Use when*: downstream task is classification/regression; you care about using LLM as a frozen encoder.
5. **LoRA / PEFT**
   - *Use when*: you have some data + a single/few GPUs and need strong domain adaptation without touching the base checkpoint.
6. **Full fine-tuning**
   - *Use when*: you own the weights, have lots of data + serious compute, and want maximum performance / deep specialization.

Architecture-wise, all these methods work for most Transformer LLMs; the **bigger drivers** are:

- how much **data** you have,
- how much **compute** and access (API vs weights),
- how far your **application domain** is from the pretraining data.

Last quick reflection (no need for a long answer):

> For your *own* future project, which method do you think you're most likely to actually use first (pure prompting, soft prompts, LoRA, etc.), and what's your main reason?

Comment 4: It provides a summary of different fine-tuning strategies and lists the reasons that depend on data access, compute resources, and distribution shift. It even affects my decision on the most appropriate fine-tuning strategy for my future project.

Prompt 5:

I think I would at first explore pure prompting due to low computational cost and generalizability

ChatGPT's answer:

Thought for 16s ›

That's a very sensible plan 👍

For most realistic projects, a good workflow is:

1. **Start with pure prompting**
   - Hand-design a clear instruction: role, task, input/output format, constraints.
   - Add **1–3 few-shot examples** if the task is a bit subtle (e.g., grading, chain-of-thought).
   - Use tools like **DSPy / GEPA / TextGrad** to *automatically* search over variants and improve the prompt on a validation set Lecture 21 .
   - This costs you almost nothing in compute and works across basically all LLM architectures.
2. **Escalate only if needed**
   - If eval shows you're still far from target on a *specialized* domain:
     - try **soft prompts / prefix tuning** when you have modest data and control over weights Lecture 21 ;
     - move to **LoRA** if you need a bigger jump;
     - consider **full fine-tuning** only when you have lots of domain data + serious GPU budget.
3. **Match method to task type**
   - **Classification / scoring** → sometimes linear probing on embeddings is easier than changing prompts.
   - **Open-ended generation / agents** → prompt engineering → soft/prefix prompts → LoRA/full FT, in that order.

Comment 5: It concludes the conversation with a nice workflow for applying fine-tuning strategies to get started on my own projects.